

INDIAN INSTITUTE OF TECHNOLOGY DHARWAD

Department of Computer Science & Engineering Assignment 3

Assignment report submitted in fulfillment of the requirements for the degree of
Master of Technology in Computer Science and Engineering

Submitted by

GROUP NO. 16

Group Members: -

Name: ADITYA NARAYAN JENA
Roll: CS26MT004

Name: KIRAN CHAKRABARTI
Roll: CS26MT023

Name: SHIVAM PATEL
Roll: CS26MT017

Under the supervision of

Prof. Dileep A. D.

Academic Year: 2026 - 2027

ABSTRACT

This report presents an experimental comparison of seven optimization algorithms for training fully connected neural networks (FCNNs) on a five-class subset of the handwritten digit dataset. Three different network depths consisting of three, four, and five hidden layers were investigated, with each depth consisting of three different hidden-node configurations. All networks employed hyperbolic tangent (tanh) activation functions in the hidden layers and cross-entropy loss for classification. Stochastic gradient descent (SGD), vanilla batch gradient descent, SGD with momentum (generalized delta rule), Nesterov Accelerated Gradient (NAG), AdaGrad, RMSProp, and Adam were evaluated under the learning-rate and batch-size specifications of the assignment. The same initial weights were used for each of the optimizers for a given architecture, and convergence was monitored using the absolute difference between successive average training errors. The results show clear differences in optimization behavior. Momentum-based methods provided a strong combination of convergence speed and validation performance for this experiment, with SGD using momentum obtaining the highest validation accuracy of 98.4453% for the four-hidden-layer architecture [128, 64, 32, 16]. The selected model achieved 99.9824% training accuracy, 98.4453% validation accuracy, and 98.1291% test accuracy. The experiments also demonstrate the aggressive learning-rate decay of AdaGrad, the improvement obtained by RMSProp through exponentially weighted gradient history, the strong performance of Adam, and the slow loss change observed with full-batch vanilla gradient descent.

Keywords— fully connected neural network, backpropagation, stochastic gradient descent, momentum, Nesterov Accelerated Gradient, AdaGrad, RMSProp, Adam, hyperbolic tangent, image classification

TABLE OF CONTENTS

I Introduction 1

II Experimental Setup 1
- II-A **Dataset and Data Partitioning** . . . 1
- II-B **Network Architectures** . . . 1
- II-C **Activation Function and Loss** . . . 1
- II-D **Optimization Algorithms** . . . 2
- II-E **Optimizer Comparison** . . . 4
- II-F **Software and Computational Environment** . . . 5
- II-G **Training Protocol and Experimental Settings** . . . 5

III Experimental Results and Discussion 5
- III-A **Convergence Epoch Comparison** . . . 5
- III-B **Training Accuracy** . . . 6
- III-C **Validation Accuracy** . . . 6
- III-D **Training-Error Curves** . . . 6
- III-E **Optimizer-wise Interpretation** . . . 11

IV Best Architecture Selection and Final Evaluation 12
- IV-A **Selection Using Validation Accuracy** . . . 12
- IV-B **Selected Model Performance** . . . 12
- IV-C **Error Analysis** . . . 13

V Discussion and Inferences 14

VI Conclusion 14

References 14

LIST OF FIGURES

1 Average training error versus epoch for 3-hidden-layer config_1 architecture. 7
2 Average training error versus epoch for 3-hidden-layer config_2 architecture. 7
3 Average training error versus epoch for 3-hidden-layer config_3 architecture. 8
4 Average training error versus epoch for 4-hidden-layer config_1 architecture. 8
5 Average training error versus epoch for 4-hidden-layer config_2 architecture. 9
6 Average training error versus epoch for 4-hidden-layer config_3 architecture. 9
7 Average training error versus epoch for 5-hidden-layer config_1 architecture. 10
8 Average training error versus epoch for 5-hidden-layer config_2 architecture. 10
9 Average training error versus epoch for 5-hidden-layer config_3 architecture. 11
10 Training confusion matrix of the selected 4-hidden-layer model. 13
11 Test confusion matrix of the selected 4-hidden-layer model. 13

LIST OF TABLES

I FCNN Architecture Configurations 1
II Convergence Epochs for All Architectures and Optimizers 5
III Final Training Accuracy (%) 6
IV Final Validation Accuracy (%) 6
V Best Validation Result at Each Network Depth 12

I. Introduction

The objective of this assignment is to study how different optimization algorithms affect the convergence and classification performance of fully connected neural networks trained using backpropagation. Unlike the previous assignment, where the emphasis was on the construction of reusable FCNN components and the effect of nonlinear activation functions, the present study focuses on the optimization mechanism and how they affect the updation of the network parameters. The assignment requires FCNNs with three to five hidden layers, different numbers of nodes in the hidden layers, and seven specified optimizers. It also requires comparison of convergence epochs, training-error curves, training and validation accuracy, and final confusion matrices for the architecture selected using validation accuracy.

The study is particularly useful because different variants of gradient descent can behave very differently even when the network architecture, initialization, learning rate, and dataset remain unchanged. Conventional gradient descent follows the current gradient directly, whereas momentum methods use information from previous updates and adaptive methods modify the effective learning rate according to gradient history. The experiment therefore provides an opportunity to relate observed training curves to the mathematical mechanisms.

II. Experimental Setup

A. Dataset and Data Partitioning

The experiments use the five-class subset of the handwritten digits dataset. The five classes are the digits 1, 5, 6, 7, and 9, which are mapped internally to class indices 0 through 4. The provided dataset contains 11,385 training images, 3,795 validation images, and 3,795 test images. Each input consists of a grayscale 28×28 pixel image, normalized to a $[0, 1]$ range, and flattened into a 784-dimensional vector prior to being fed into the fully connected neural network (FCNN). The class mapping was verified to be identical across the training, validation, and test partitions.

B. Network Architectures

Three network depths were considered: three, four, and five hidden layers. For each depth, three decreasing-width hidden-node configurations were evaluated. The final layer of each configuration contains five output units corresponding to the five classes of the digits. All hidden layers used hyperbolic tangent as activation function.

TABLE I
FCNN Architecture Configurations

Hidden layers	Configuration	Hidden-layer nodes
3	config_1	128–64–32
3	config_2	256–128–64
3	config_3	512–256–128
4	config_1	128–64–32–16
4	config_2	256–128–64–32
4	config_3	512–256–128–64
5	config_1	128–64–32–16–8
5	config_2	256–128–64–32–16
5	config_3	512–256–128–64–32

C. Activation Function and Loss

The hyperbolic tangent (tanh) activation function was used in every hidden layer of all FCNN architectures. For an activation input a, the tanh function is defined as

$$\tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}. \tag{1}$$

Its derivative, which is required during backpropagation, is

$$\frac{d\tanh(a)}{da} = 1 - \tanh^{2}(a). \tag{2}$$

The tanh activation introduces the nonlinear transformation required for the FCNN to learn nonlinear decision boundaries. During backpropagation, its derivative determines how the error gradient is propagated through the hidden layers.

For the classification objective, cross-entropy loss was used. For a sample with target class y and output logits $z_1, z_2, \ldots, z_K$, the cross-entropy loss can be expressed as

$$\mathcal{L} = -\log\left(\frac{e^{z_y}}{\sum_{k=1}^{K} e^{z_k}}\right). \tag{3}$$

The final layer produces five class logits corresponding to the five classes. No explicit softmax layer was added. The PyTorch `CrossEntropyLoss` function was applied directly to these logits and internally performs the log-softmax operation required for the cross-entropy objective. The predicted class is obtained by selecting the output neuron with the largest logit.

D. Optimization Algorithms

Let $w(m)$ denote a weight parameter at the m-th optimization iteration and let $\Delta w(m)$ denote the corresponding change in the weight. The gradient of the error with respect to the weight is denoted by

$$\nabla_w(m) = \frac{\partial E(m)}{\partial w(m)}, \tag{4}$$

where $E(m)$ represents the error at the current iteration. The learning rate is denoted by η.

The basic gradient-descent update is

$$\begin{aligned} \Delta w(m) &= -\eta \nabla_w(m), \\ w(m+1) &= w(m) + \Delta w(m). \end{aligned} \tag{5}$$

The seven optimization algorithms studied in this experiment modify this basic update by either using the current gradient, incorporating previous weight changes, evaluating the gradient at a look-ahead point, or adapting the effective learning rate using gradient history.

*1) **Stochastic Gradient Descent (SGD):*** Stochastic gradient descent updates the network parameters using the gradient calculated from a single training example. Unlike batch gradient descent, the parameters are updated after every training pattern. Therefore, for a training set containing N examples, one epoch contains N parameter updates when the batch size is one.

The update is

$$\begin{aligned} \Delta w(m) &= -\eta \nabla_w E_n(m), \\ w(m+1) &= w(m) + \Delta w(m), \end{aligned} \tag{6}$$

where $E_n(m)$ represents the error associated with the current training example.

In the present experiment,

$$\eta = 0.001, \qquad B = 1. \tag{7}$$

Therefore, SGD performs 11,385 parameter updates during each epoch. The use of a single training example produces a noisy gradient estimate, which can cause oscillations, but the frequent updates allow the optimizer to move rapidly through the parameter space.

*2) **Vanilla Batch Gradient Descent:*** Vanilla batch gradient descent computes the gradient using the complete training dataset before updating the network parameters. Thus, one epoch produces one optimizer update.

The average gradient over the complete training set can be represented as

$$\nabla_w E_{\text{avg}}(m) = \frac{1}{N} \sum_{n=1}^{N} \nabla_w E_n(m). \tag{8}$$

The corresponding update is

$$\begin{aligned} \Delta w(m) &= -\eta \nabla_w E_{\text{avg}}(m), \\ w(m+1) &= w(m) + \Delta w(m). \end{aligned} \tag{9}$$

For this experiment,

$$B = N = 11385, \qquad \eta = 0.001. \tag{10}$$

Since the complete training set is processed before each update, only one parameter update is performed per epoch. This provides a stable gradient estimate but can result in slow parameter movement when the learning rate is small.

*3) **Generalized Delta Rule**:* The generalized delta rule incorporates information from the previous weight update into the current update. The momentum term helps reinforce movement when successive gradients point in a similar direction and reduces abrupt changes in the optimization trajectory.

The update equation used in the lecture formulation is

$$\Delta w(m) = -\eta \nabla_w(m) + \alpha \Delta w(m-1), \qquad 0 < \alpha < 1. \tag{11}$$

The new weight is then obtained as

$$w(m+1) = w(m) + \Delta w(m). \tag{12}$$

The parameters used in the experiment were

$$\eta = 0.001, \qquad \alpha = 0.9, \qquad B = 1. \tag{13}$$

The momentum mechanism resulted in strong convergence behaviour across the architectures. It achieved the highest validation accuracy in the complete experiment, reaching 98.4453% for the four-hidden-layer architecture with hidden nodes [128,64,32,16].

*4) **Nesterov Accelerated Gradient (NAG)**:* Nesterov Accelerated Gradient extends the momentum approach by evaluating the gradient at a look-ahead position. Instead of calculating the gradient only at the current weight, the optimizer first estimates the position toward which momentum will move the parameters.

The look-ahead weight is defined as

$$w_{\text{lookahead}} = w(m) + \alpha \Delta w(m-1). \tag{14}$$

The gradient is then evaluated at this look-ahead position and used in the update:

$$\begin{aligned} \Delta w(m) &= \alpha \Delta w(m-1) - \eta \nabla_{w_{\text{lookahead}}} E(m), \\ w(m+1) &= w(m) + \Delta w(m). \end{aligned} \tag{15}$$

The parameters used were

$$\eta = 0.001, \qquad \alpha = 0.9, \qquad B = 1. \tag{16}$$

The look-ahead mechanism allows NAG to correct the direction of movement before the parameter reaches the predicted position. This can reduce oscillations and improve convergence compared with conventional momentum.

*5) **AdaGrad**:* AdaGrad introduces a separate adaptive learning rate for each parameter by maintaining a cumulative history of squared gradients. For the l-th parameter, the accumulated squared-gradient history is

$$v_l(m) = v_l(m-1) + \left(\nabla_{w_l}(m)\right)^2. \tag{17}$$

The corresponding parameter update is

$$w_l(m+1) = w_l(m) - \frac{\eta \nabla_{w_l}(m)}{\sqrt{v_l(m)} + \epsilon}. \tag{18}$$

Here, ϵ is a small positive constant used for numerical stability. The parameters used in this experiment were

$$\eta = 0.001, \qquad \epsilon = 10^{-8}, \qquad B = 11385. \tag{19}$$

Because the squared gradients are accumulated throughout training, the value of $v_l(m)$ generally increases over time. Consequently, the effective learning rate can decrease substantially. This behaviour explains the comparatively large convergence epochs observed for AdaGrad in the experiments. For example, the five-hidden-layer configuration [128,64,32,16,8] required 2,200 epochs to satisfy the convergence criterion.

*6) **RMSProp**:* RMSProp modifies the AdaGrad approach by replacing the cumulative squared-gradient history with an exponentially weighted moving average. Thus, older gradients gradually lose their influence.

The moving average of squared gradients is given by

$$v_l(m) = \beta v_l(m-1) + (1-\beta)\left(\nabla_{w_l}(m)\right)^2. \tag{20}$$

The corresponding parameter update is

$$w_l(m+1) = w_l(m) - \frac{\eta \nabla_{w_l}(m)}{\sqrt{v_l(m)} + \epsilon}. \tag{21}$$

The parameters used in the experiment were

$$\eta = 0.001, \qquad \beta = 0.99, \qquad \epsilon = 10^{-8}, \qquad B = 11385. \tag{22}$$

Unlike AdaGrad, the exponential moving average prevents the update-history term from growing indefinitely. Therefore, RMSProp avoids the excessively rapid learning-rate decay associated with cumulative gradient accumulation and generally converges faster than AdaGrad in the experiments.

*7) **Adam**:* Adam combines the advantages of momentum-based optimization and adaptive learning-rate scaling. It maintains two exponentially weighted moving averages: the first moment of the gradient and the second moment of the squared gradient.

The first moment is calculated as

$$u_l(m) = \beta_1 u_l(m-1) + (1-\beta_1)\nabla_{w_l}(m). \tag{23}$$

The second moment is

$$v_l(m) = \beta_2 v_l(m-1) + (1-\beta_2)\left(\nabla_{w_l}(m)\right)^2. \tag{24}$$

Since both moment estimates are initialized to zero, bias correction is applied:

$$\begin{aligned} \hat{u}_l(m) &= \frac{u_l(m)}{1-\beta_1^m}, \\ \hat{v}_l(m) &= \frac{v_l(m)}{1-\beta_2^m}. \end{aligned} \tag{25}$$

The final Adam parameter update is

$$w_l(m+1) = w_l(m) - \eta \frac{\hat{u}_l(m)}{\sqrt{\hat{v}_l(m)} + \epsilon}. \tag{26}$$

The parameters used in the experiment were

$$\eta = 0.001, \qquad \beta_1 = 0.9, \qquad \beta_2 = 0.999, \qquad \epsilon = 10^{-8}, \qquad B = 1. \tag{27}$$

The first moment provides information about the direction of parameter movement, while the second moment provides parameter-wise scaling of the update. This combination resulted in rapid convergence for several configurations. The fastest Adam convergence occurred at 9 epochs for the five-hidden-layer configuration [512,256,128,64,32], while its highest validation accuracy was 98.2609%.

E. Optimizer Comparison

The seven optimizers can therefore be viewed as different modifications of the basic gradient-descent update. SGD uses the current stochastic gradient directly, whereas vanilla gradient descent uses the gradient obtained from the complete training set. The generalized delta rule incorporates previous weight changes, while NAG evaluates the gradient at a look-ahead position before applying the momentum update. AdaGrad introduces a cumulative squared-gradient history, whereas RMSProp replaces this cumulative history with an exponentially weighted moving average. Adam combines first- and second-moment estimates with bias correction. These differences in update mechanisms form the basis for interpreting the differences observed in convergence epochs, training-error curves, and classification accuracy.

F. *Software and Computational Environment*

The experiments were implemented in Python using the PyTorch deep learning framework. PyTorch was used for constructing fully connected neural networks, tensor operations, automatic differentiation, loss computation, and optimization. The torchvision library was used for loading the image dataset and applying image transformations. NumPy was used for numerical operations and data handling, while Matplotlib was used to generate the training-error plots and visualizations. JSON was used to store and retrieve optimizer-wise experimental results. Python's random module and PyTorch random-seed functions were used to maintain reproducibility.

Library	**Purpose**
PyTorch	FCNN construction, tensors, automatic differentiation, loss computation and optimization
torchvision	Dataset loading and image preprocessing
NumPy	Numerical operations and data handling
Matplotlib	Training-error plots and visualizations
JSON	Saving and loading experimental results
Python random	Reproducible randomization

The implementation uses PyTorch's `torch.nn`, `torch.utils.data`, and `torch.optim` modules for the neural network, data loading, and optimizer implementations, respectively.

G. *Training Protocol and Experimental Settings*

All experiments were performed using the same five-class dataset and the same set of FCNN architecture configurations for every optimizer. To ensure a fair comparison, the same saved initial model state was loaded for all optimizers corresponding to a particular architecture. The training data were shuffled using a fixed random seed.

The learning rate was set to $\eta = 0.001$ for all optimizers. SGD, generalized delta rule, NAG, and Adam used batch size 1, whereas vanilla gradient descent, AdaGrad, and RMSProp used the complete training set of 11,385 examples as a single batch. The momentum parameter was set to $\alpha = 0.9$ for generalized delta rule and NAG. RMSProp used $\beta = 0.99$ and $\epsilon = 10^{-8}$, while Adam used $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

The average training error was recorded after every completed epoch. Training was considered converged when the absolute difference between the average training errors of two successive epochs became smaller than 10^{-4}. In the implementation, the convergence test was deliberately disabled during the first two completed epochs. This minimum warm-up prevents an optimizer from terminating immediately because of a negligible initial loss change. After the warm-up period, the specified 10^{-4} loss-difference threshold was used as the convergence criterion.

III. Experimental Results and Discussion

A. *Convergence Epoch Comparison*

Table II summarizes the number of epochs required by each optimizer to satisfy the convergence criterion. The results show a clear distinction between momentum-based, adaptive, and conventional gradient methods. Momentum and NAG generally converge within a few tens of epochs, whereas AdaGrad requires hundreds or thousands of epochs. RMSProp is substantially faster than AdaGrad but remains slower than the momentum-based methods for most configurations. Adam is also highly competitive. Vanilla gradient descent shows two very different behaviors: it terminates very early for most configurations because its full-batch updates change the loss only slightly, while the wider three-hidden-layer configuration requires 4,838 epochs before satisfying the threshold.

TABLE II
Convergence Epochs for All Architectures and Optimizers

Depth	Config.	SGD	Vanilla GD	Momentum	NAG	AdaGrad	RMSProp	Adam
3	config_1	53	3	22	24	929	116	85
3	config_2	41	3	25	16	610	154	78
3	config_3	44	4838	24	24	345	261	54
4	config_1	31	3	25	28	1872	50	126
4	config_2	57	3	27	28	915	155	62
4	config_3	64	3	29	30	284	134	41
5	config_1	31	1	46	8	2200	254	100
5	config_2	52	1	28	30	1785	54	22
5	config_3	41	1	38	36	309	79	9

B. *Training Accuracy*

Table III presents the training accuracies of the three architectures across three distinct configurations for all seven optimizers.

TABLE III
FINAL TRAINING ACCURACY (%)

Depth	Config.	SGD	Vanilla GD	Momentum	NAG	AdaGrad	RMSProp	Adam
3	config_1	99.9912	19.9385	100.0000	100.0000	96.5657	98.0061	99.9297
3	config_2	99.8419	19.7277	99.9912	99.9561	97.3298	98.5419	99.9824
3	config_3	99.8946	91.9368	100.0000	100.0000	97.8305	99.4027	99.8682
4	config_1	98.0325	20.0000	99.9824	99.9824	98.0764	96.1177	100.0000
4	config_2	99.9736	19.9912	100.0000	100.0000	98.2257	98.5332	99.8243
4	config_3	99.9824	20.0000	100.0000	100.0000	97.4089	98.3663	99.9209
5	config_1	99.7189	20.0000	99.9824	98.9284	98.3311	99.3412	100.0000
5	config_2	99.9473	20.0000	99.9034	99.9912	99.3237	93.4124	99.3061
5	config_3	99.6487	20.0000	100.0000	99.9561	97.4264	97.1981	98.9284

C. *Validation Accuracy*

Table IV presents the validation accuracies of the three architectures across three distinct configurations for all seven optimizers.

TABLE IV
FINAL VALIDATION ACCURACY (%)

Depth	Config.	SGD	Vanilla GD	Momentum	NAG	AdaGrad	RMSProp	Adam
3	config_1	97.7075	20.0000	98.3663	98.2082	95.9157	96.8116	98.1818
3	config_2	97.3913	19.6838	97.8920	98.0764	96.2055	97.0487	98.2345
3	config_3	97.5231	92.7536	98.0501	97.9447	96.4954	97.3123	98.0237
4	config_1	95.9420	20.0000	98.4453	98.2609	96.8116	95.5468	98.2609
4	config_2	97.3650	20.0000	98.0764	98.0764	96.7589	97.0487	97.8920
4	config_3	97.3913	20.0000	97.9974	98.0237	96.2319	97.0487	98.2609
5	config_1	97.2859	20.0000	98.0764	97.5758	96.8116	97.4967	98.0764
5	config_2	97.5231	20.0000	97.8920	98.2082	97.2332	93.4387	97.6021
5	config_3	97.0487	20.0000	98.1818	98.2082	96.0211	96.3373	97.6548

D. *Training-Error Curves*

The training-error plots below superimpose the seven optimizer trajectories for each architecture configuration. The curves make the convergence differences visible beyond the single convergence-epoch value. SGD with momentum and NAG generally descend rapidly because the update direction incorporates recent parameter movement. Adam also descends rapidly in most cases because it combines a momentum-like first moment with adaptive scaling based on the second moment. AdaGrad descends much more gradually as its cumulative squared-gradient history continuously reduces the effective learning rate. RMSProp mitigates this effect by using an exponentially weighted history. Vanilla gradient descent remains near its initial error for many of the configurations, reflecting the small update obtained from one full-batch step per epoch.

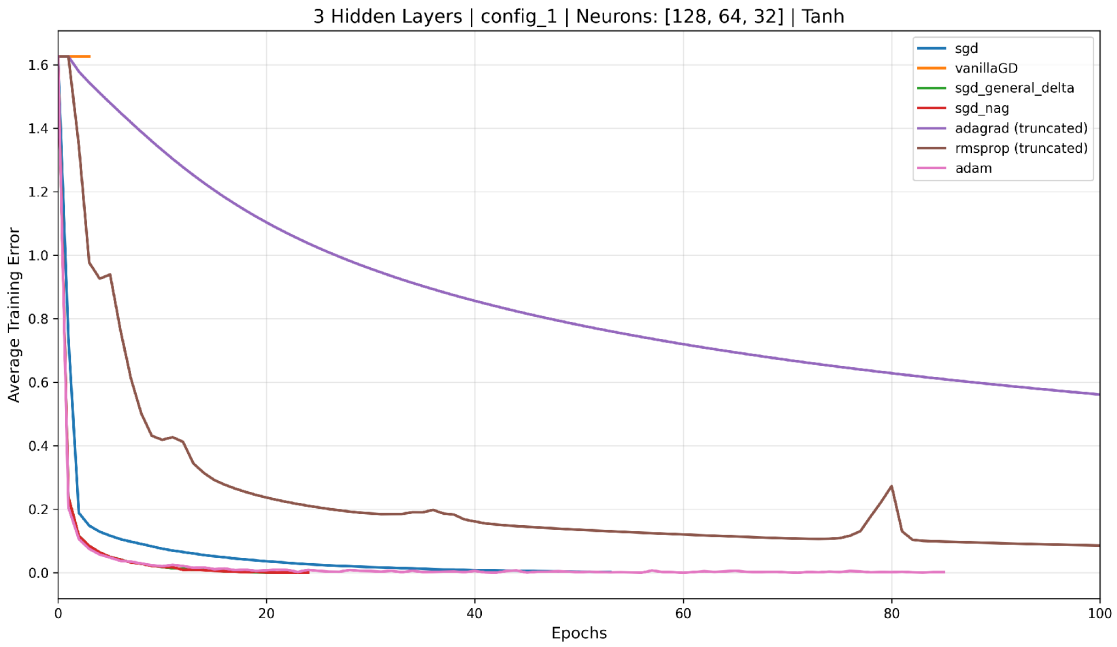


Fig. 1. Average training error versus epoch for 3-hidden-layer config_1 architecture.

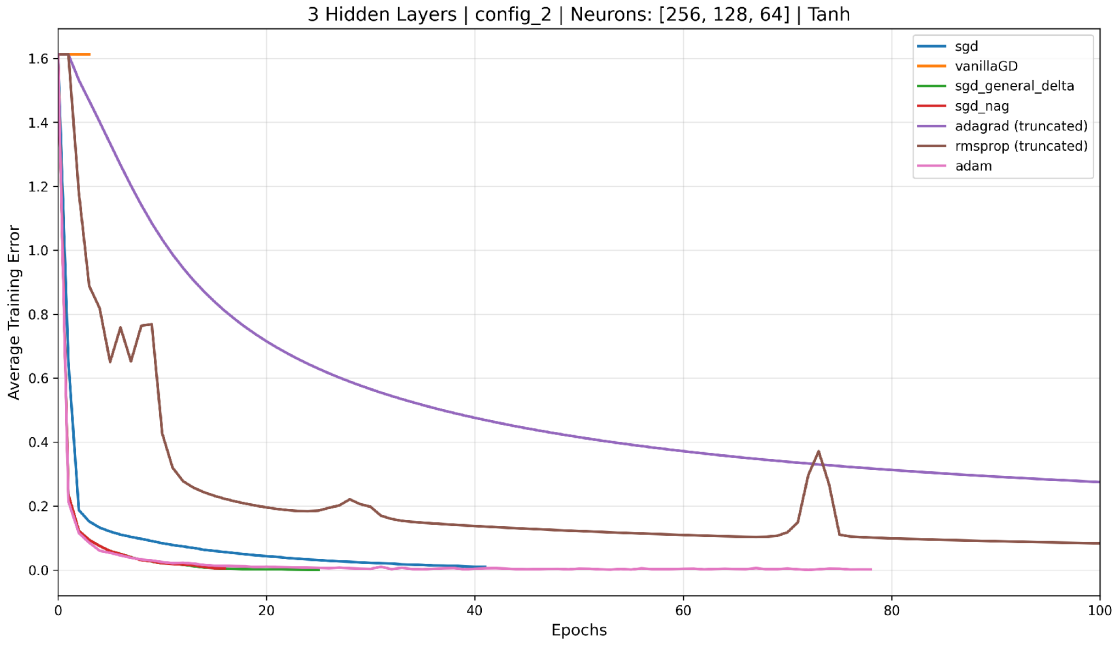


Fig. 2. Average training error versus epoch for 3-hidden-layer config_2 architecture.

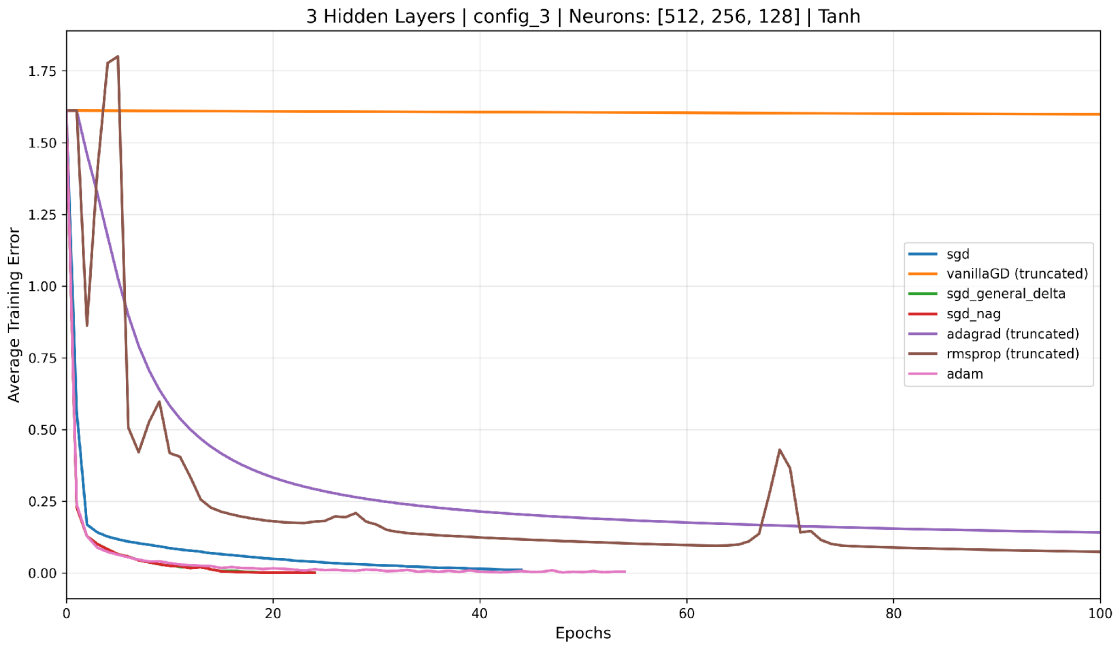


Fig. 3. Average training error versus epoch for 3-hidden-layer config_3 architecture.

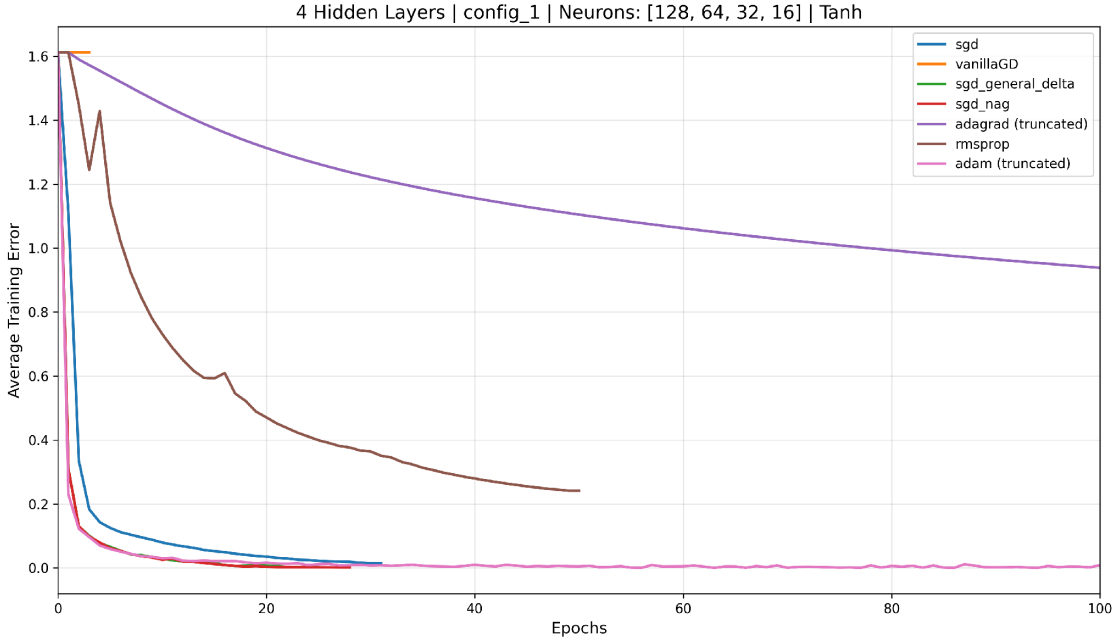


Fig. 4. Average training error versus epoch for 4-hidden-layer config_1 architecture.

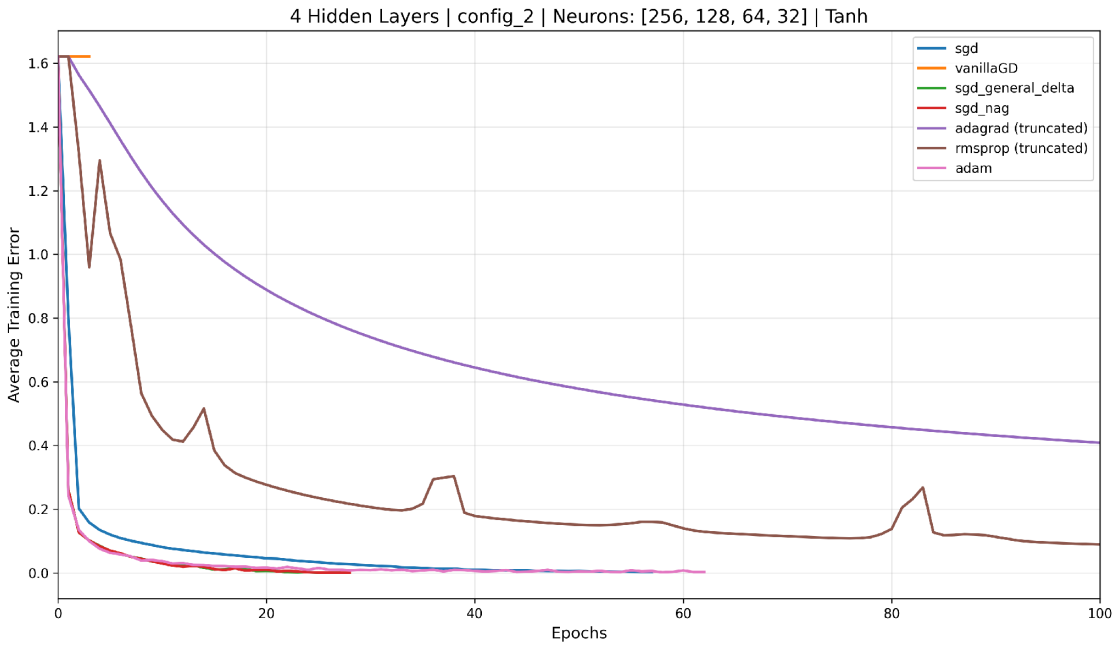


Fig. 5. Average training error versus epoch for 4-hidden-layer config_2 architecture.

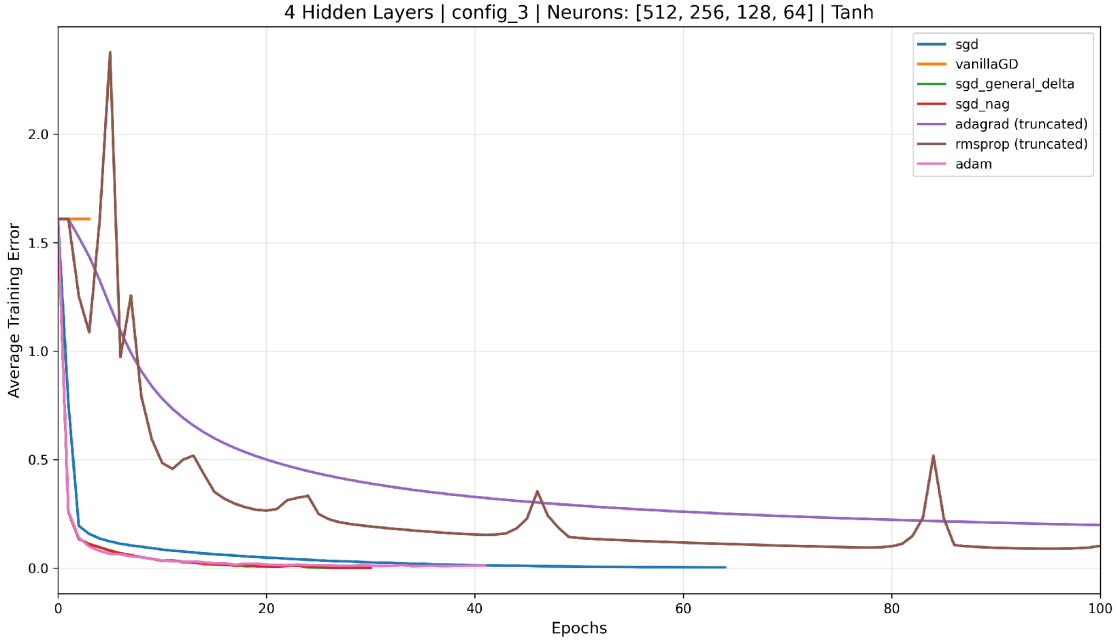


Fig. 6. Average training error versus epoch for 4-hidden-layer config_3 architecture.

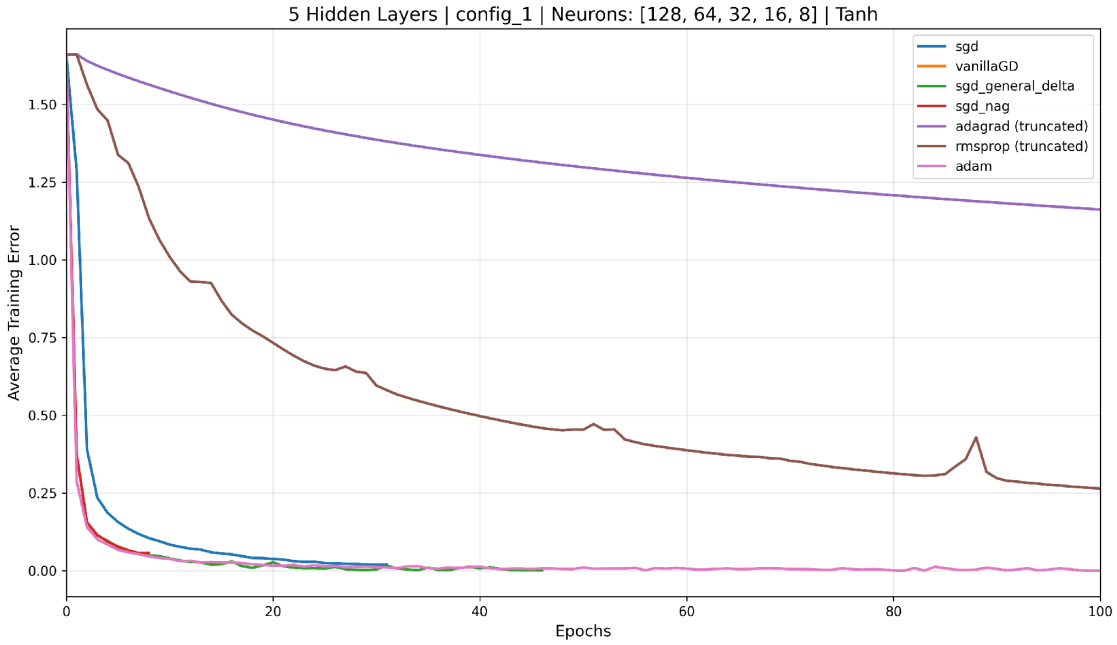


Fig. 7. Average training error versus epoch for 5-hidden-layer config_1 architecture.

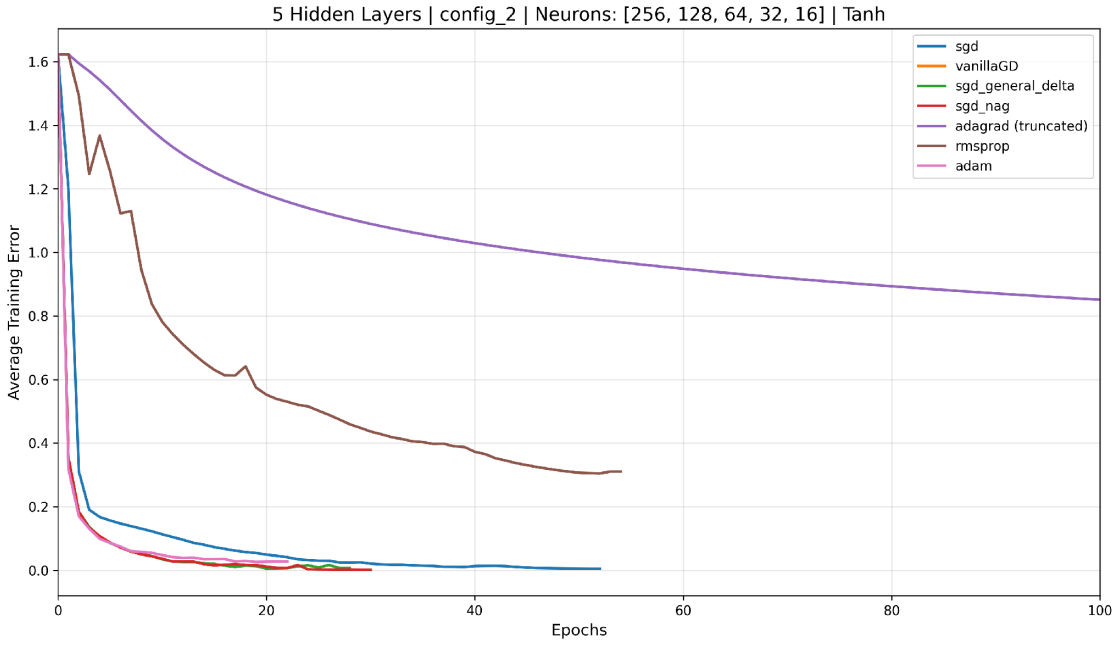


Fig. 8. Average training error versus epoch for 5-hidden-layer config_2 architecture.

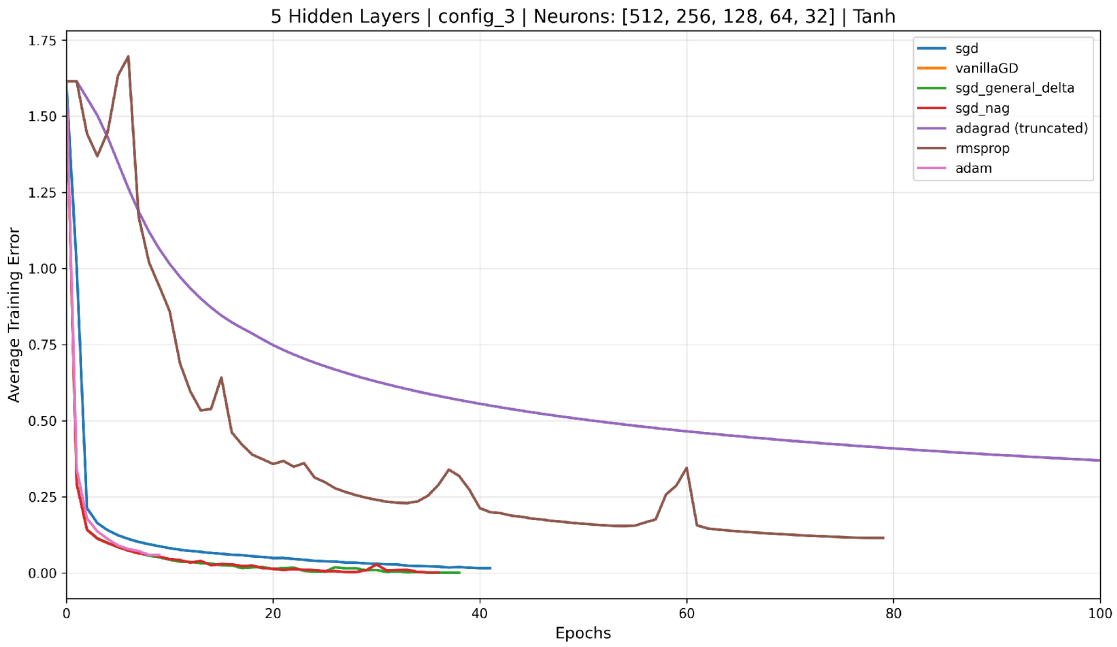


Fig. 9. Average training error versus epoch for 5-hidden-layer config_3 architecture.

E. Optimizer-wise Interpretation

SGD: With batch size 1, SGD performs an update after every training pattern. Consequently, each epoch contains 11,385 parameter updates. This produces a noisy but frequently effective optimization trajectory and allows the model to move substantially even when the full-batch gradient would be small. The results show good final accuracy, although momentum-based variants generally achieve higher validation accuracy and faster convergence.

Vanilla Gradient Descent: Vanilla GD uses the entire training set for every update, resulting in only one parameter update per epoch. The loss therefore changes very slowly in several architectures. Because the stopping rule is based on the difference between successive average errors rather than classification accuracy, a nearly unchanged loss can satisfy the threshold even when the classifier remains close to chance performance. For the five-class problem, the approximately 20% accuracy observed in several runs corresponds to chance-level classification. The 3-hidden-layer configuration [512, 256, 128] is an exception: it continues for 4,838 epochs indicating full batch gradient descent takes higher epochs to converge and reaches 92.7536% validation accuracy.

Generalized Delta Rule: The momentum update augments the current gradient with a fraction of the previous parameter update. When consecutive gradients point in a consistent direction, the history term reinforces the movement and accelerates progress through shallow regions of the loss surface. This behavior is strongly reflected in the experiments. Generalized delta produced the best overall validation accuracy: 98.4453% for the four-hidden-layer [128, 64, 32, 16] network.

NAG: NAG uses momentum but evaluates the gradient at a look-ahead position. This gives the optimizer information about the direction in which the momentum term is already moving the parameters and allows earlier correction of the trajectory. It therefore converges particularly quickly for several deeper configurations. Its best validation accuracy was 98.2082% for the five-hidden-layer configurations [256, 128, 64, 32, 16] and [512, 256, 128, 64, 32].

AdaGrad: AdaGrad assigns different effective learning rates to parameters using the accumulated history of squared gradients. This is useful when parameters have sparse features, but the cumulative history keeps increasing. Consequently, the effective learning rate continuously decreases. The experiments show this clearly: AdaGrad requires as many as 2,200 epochs for the five-hidden-layer [128, 64, 32, 16, 8] configuration. The validation accuracy of the Adagrad optimizer remains consistently below the peak values achieved by the momentum and Adam optimizers, indicating premature convergence before reaching the optimal minimum on the loss curve.

RMSProp: RMSProp addresses the main limitation of AdaGrad by replacing the unbounded cumulative history with an exponentially weighted moving average of squared gradients. Old gradients gradually lose influence, preventing the effective learning rate from shrinking as aggressively. This explains why RMSProp converges

substantially faster than AdaGrad in the present experiments. Nevertheless, its full-batch training mode means that it still performs only one optimizer update per epoch.

Adam: Adam combines a first-moment estimate of the gradient with a second-moment estimate of the squared gradient and applies bias correction. It therefore combines momentum-like directional information with parameter-wise adaptive scaling. Adam achieves consistently high validation accuracy and very fast convergence for several architectures, including only 9 epochs for the five-hidden-layer [512, 256, 128, 64, 32] configuration. However, its best validation accuracy of 98.2609% is slightly below the 98.4453% obtained by generalized delta in the selected four-layer architecture. Thus, under the fixed conditions of this experiment, Adam was highly competitive but not the best-performing optimizer.

IV. Best Architecture Selection and Final Evaluation

A. *Selection Using Validation Accuracy*

The best architecture was selected using validation accuracy as required by the assignment. The best result for three hidden layers was obtained by generalized delta rule with configuration [128, 64, 32], giving 98.3663% validation accuracy. For four hidden layers, generalized delta rule with [128, 64, 32, 16] achieved 98.4453%. For five hidden layers, NAG achieved 98.2082% with both [256, 128, 64, 32, 16] and [512, 256, 128, 64, 32]. Therefore, the four-hidden-layer configuration [128, 64, 32, 16] trained with generalized delta was selected.

TABLE V
Best Validation Result at Each Network Depth

Depth	Best configuration	Optimizer	Validation accuracy (%)
3	[128,64,32]	Generalized Delta	98.3663
4	[128,64,32,16]	Generalized Delta	98.4453
5	[256,128,64,32,16]	NAG	98.2082

B. *Selected Model Performance*

The selected network has the architecture 784-128-64-32-16-5, uses tanh in the hidden layers, and is trained using SGD with momentum (generalized delta rule), batch size 1, learning rate 0.001, and momentum 0.9. It converged after 25 epochs. The final training accuracy was 99.9824%, while validation accuracy was 98.4453%.

The training confusion matrix contains only two errors, resulting in 99.9824% training accuracy. On the independent test set, the model correctly classified 3,724 of 3,795 images, giving a test accuracy of 98.1291%.

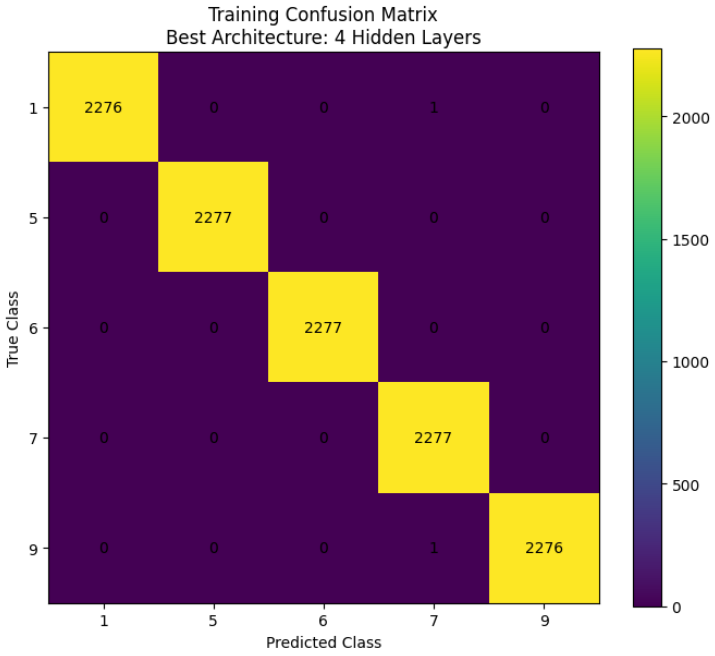


Fig. 10. Training confusion matrix of the selected 4-hidden-layer model.

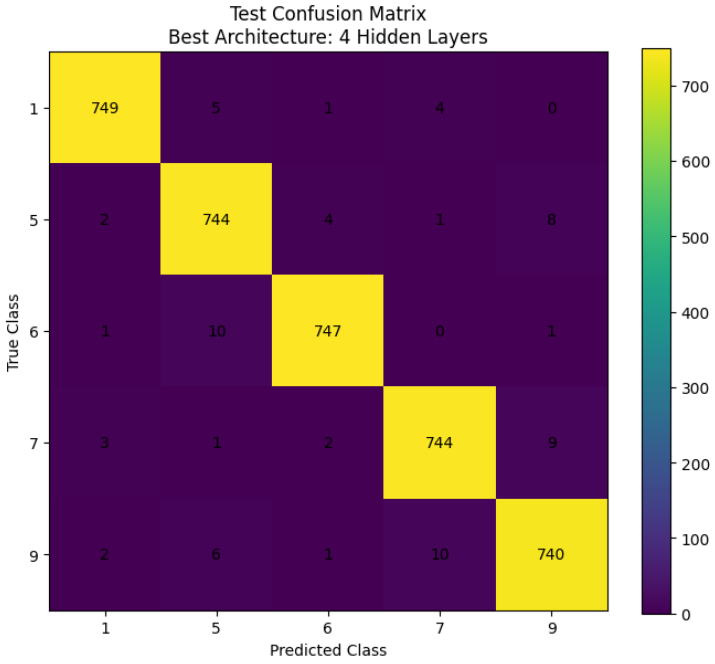


Fig. 11. Test confusion matrix of the selected 4-hidden-layer model.

C. *Error Analysis*

The test confusion matrix shows that the residual errors are concentrated among a small number of visually similar handwritten patterns. The largest off-diagonal counts include class 6 being predicted as class 5 in 10 cases, class 9 being predicted as class 7 in 10 cases, class 7 being predicted as class 9 in 9 cases, and class 5 being predicted as class 9 in 8 cases. Despite these errors, the diagonal remains dominant for every class, indicating strong generalization from the training data to unseen examples.

V. Discussion and Inferences

The experiments demonstrate that optimizer performance cannot be judged from convergence epoch alone. A smaller epoch count is useful only when the stopping point corresponds to a meaningful reduction in classification error. This is particularly evident for vanilla gradient descent, where a minor reduction in average loss can satisfy the convergence threshold while the classification accuracy remains close to the five-class chance level. Conversely, the momentum-based methods generally combine rapid loss reduction with high validation accuracy.

The comparison also shows that greater network depth does not automatically produce better validation performance. The best from three-hidden-layer, four-hidden-layer, and five-hidden-layer results were 98.3663%, 98.4453%, and 98.2082%, respectively. The four-hidden-layer architecture therefore provided the best balance for the present dataset and fixed training settings. Increasing the depth to five hidden layers did not produce a corresponding improvement in generalization.

Another important observation is the difference between full-batch and pattern-mode training. Batch size 1 permits thousands of parameter updates during a single epoch, whereas full-batch methods make one optimizer update after processing all 11,385 training examples. Thus, the reported epoch counts should be interpreted together with batch size. In this experiment, the momentum-based pattern-mode methods benefited from frequent updates and historical direction information, while the full-batch methods were more sensitive to small gradients and learning-rate adaptation.

The strong performance of generalized delta is consistent with the mathematical purpose of momentum described in the course notes: previous parameter changes reinforce a consistent descent direction and help the optimizer move more effectively through gentle-gradient regions [2]. NAG applies a related idea with a look-ahead gradient, while RMSProp modifies the effective learning rate using gradient-history statistics and Adam combines look-ahead gradient along with effective learning rate modification using gradient history. Hence, the resulting performance ranking is systematic rather than arbitrary; it demonstrates the unique sensitivity of each optimization algorithm to the gradient landscape produced by the five-class image-classification task.

VI. Conclusion

This study compared seven optimization algorithms across nine fully connected neural-network architectures for five-class digit classification. The experiments showed substantial differences in convergence behavior and final classification performance even when the same initial weights and common learning-rate settings were used. Momentum-based methods provided the strongest overall results, with generalized delta rule stochastic gradient descent achieving the highest validation accuracy of 98.4453% using the four-hidden-layer [128, 64, 32, 16] architecture.

The selected model achieved 99.9824% training accuracy, 98.4453% validation accuracy, and 98.1291% test accuracy. AdaGrad required substantially more epochs because its cumulative squared-gradient history progressively reduced the effective learning rate, while RMSProp mitigated this effect through exponential averaging. Adam provided fast convergence and consistently high accuracy by combining first-moment and second-moment information with bias correction. NAG also converged rapidly because of its look-ahead mechanism.

The vanilla gradient descent results highlighted an important distinction between numerical convergence and useful classification learning. A small change in average loss can satisfy the prescribed convergence threshold without producing a well-trained classifier, particularly when full-batch updates are combined with small propagated gradients. Overall, the experimental study confirms that optimizer choice, batch size, network depth, and gradient behavior jointly determine the effectiveness of backpropagation for this classification task.

References

[1] CS601T: Deep Learning, "Programming Assignment 3," Department of Computer Science and Engineering, Indian Institute of Technology Dharwad.

[2] CS601T: Deep Learning, "Optimization Methods for Backpropagation," Course Lecture Notes, Indian Institute of Technology Dharwad.